

Campminder — Senior AI Platform Engineer: positioning notes

Companion to forest-yox-resume-campminder.md. Not for submission.

The arc the recruiter told us to hit

Nick's general feedback: *"Ideal candidate was someone who previously was a SWE delivering customer facing products and has more recently transitioned into an AI focused role that focuses on enablement/tooling."*

That is the story the revised resume tells, in this order:

- 1 A decade of shipping customer-facing production software (DISH support tooling, BryterCX dashboards, Volume Media React/React Native apps).
- 2 Customer-facing **AI** delivery at Soul Machines — Digital Personas in front of enterprise clients, with you personally in the room for the AI interactions.
- 3 A recent pivot toward AI **enablement and tooling** — agentic coding at Volume Media, plus the Fleet testbed where you build rules, skills, subagent workflows, and MCP integrations.

What changed and why

Change	Reason
Added the headline "Senior Engineer — AI Platform, Tooling & Enablement"	The old resume opened with "Front-end lead engineer," which reads as a front-end specialist, not an AI platform candidate.
Rewrote the summary around the SWE-to-AI arc	Mirrors the recruiter's ideal-candidate profile in the first four lines.
Moved AI to the top of Technical Skills	Recruiters and ATS keyword scans read top-down; AI was previously buried in a "Specialized" line.
Promoted the coding-agents bullet to first at Volume Media	It was the seventh bullet. It is now your most relevant experience for this role.
Expanded Soul Machines significantly	Nick killed Candidate 2 for having "the AI experience but not the experience delivering solutions that are customer facing." This block is your answer to that.
Added the Fleet project section	The JD explicitly asks for experience "extending or building on agentic coding tools (Claude Code, Cursor, MCPs, custom agents, skills/plugins)." Fleet is direct evidence, and it is checkable.
Reframed Technergetics image-annotation packages as ML data supply chain	Same work, stated in language that connects to AI.
Reframed enablement-flavored bullets (Storybook, Connections Team, mentoring, client demos)	"How to enable people internally" is a listed nice-to-have and half of the team's mission.

Fill these in before you send it

The resume has bracketed placeholders. **Use real numbers or delete the clause** — everything on this resume will be probed across five interview rounds, including one with the CTO.

- [X%] — feature turnaround improvement from agentic coding at Volume Media. If you never measured it, say something you can defend instead ("across [N] feature branches") or cut it.
- [N] users — scale of the Volume Media React/React Native apps.

Claims to sanity-check against your own history

I reframed your existing bullets; I did not invent employers, titles, dates, or projects. But two lines lean on things only you can confirm:

- 1 **The AI observability clause in Technical Skills** (prompt/response logging, token-spend tracking, regression evals). Nick flagged Candidate 3's observability gap as "significant for this role," so it is tempting to leave in — but the JD lists it as its own bullet, which means someone will ask. If you have not built it, cut it from Skills and use the honest version in the Question 2 answer below, where a gap becomes a lesson.
- 2 **"Brought agentic coding into the team's workflow"** at Volume Media. Your original bullet framed this as increasing *your own* efficiency. If it stayed personal, change this to "Adopted agentic coding in a production codebase" and keep the team-wide version for the interview as what you would do next.

Interview prep — the four questions Nick sent

Nick's coaching points show what sank the other three candidates: no business metrics, no convincing "wish I'd done better," no customer-facing delivery, execution-level rather than direction-setting thinking, and an AI observability gap. Every answer below is built to close one of those.

1. Difficulty communicating with an executive

Use: the Volume Media hackathon project that became a production feature. Backup: the Technergetics proof-of-concept demos that won contracts.

The shape that works: your first pitch led with the implementation — the model, the architecture, how it worked. Leadership had no way to evaluate that, so it stalled. You re-cut the pitch to open with the business outcome and the metric it would move, showed a working demo instead of a diagram, and kept the technical detail in reserve for whoever asked. It got funded and shipped.

Name the actual metric. That is precisely where Candidate 1 lost the room.

2. Two production AI tools, and what you'd do differently

Tool 1 — Soul Machines Digital Personas (customer-facing). Problem: enterprise clients wanted brand interactions that felt human rather than scripted. Why AI was right: open-ended conversation cannot be enumerated as deterministic flows, and the avatar's value depended on responding naturally in real time. Constraint that defined the work: perceptible latency — past a certain delay the persona stops feeling human, so latency was a product requirement, not an optimization.

What you would do differently — **lead with observability:** prompt/input/output logging, per-session token and cost tracking, and a regression eval suite for persona behavior, built from day one instead of debugging live on client calls. This answers the exact gap Nick flagged, and it is credible because you felt the absence of it.

Tool 2 — agentic coding at Volume Media (internal). Problem: engineering throughput lost to boilerplate and routine implementation. Why AI was right: that code is pattern-heavy, low-novelty, and cheaply verifiable against an existing Jest/Cypress and Storybook harness — a fast feedback loop is what makes model output safe to accept. Show product judgment by naming where you deliberately kept deterministic code.

What you would do differently: **measure a baseline first** (cycle time, PR review time, escaped defects) so the gain is arguable rather than anecdotal, and ship it as shared team tooling — rules, skills, and review conventions — rather than a personal workflow. That is the direction-setting framing Candidate 3 lacked.

3. Staying current, and a recent experiment

Use Fleet. It is a genuine, current, inspectable experiment: a real app built specifically to test how far agentic workflows can be pushed.

What you learned is the interesting part, and it maps onto their mission almost line for line: prompting the same standard repeatedly does not scale, so you encoded standards as always-on rules; repeatable multi-step work (adding a table with RLS, types, and a CRUD slice) belongs in a skill rather than a prompt; and broad exploration is better delegated to a subagent than crammed into one context window. That is the difference between using AI tools and building a platform on them — which is the job.

4. Helping a non-technical person adopt AI

Use the Soul Machines client stakeholders. They were non-technical, they were adopting a conversational AI product, and you ran those interactions.

The barrier was trust, not usability: they could not predict what the persona would say, so they wanted to over-script it, which would have destroyed the thing they were buying. What worked was showing failure modes openly instead of demoing only the happy path, narrowing initial scope to conversations where being wrong was cheap, and giving them a clear mental model of what the model could and could not be relied on for. Confidence followed from understanding the boundaries.

Backup, if they want an internal example: onboarding non-engineers to review UI in Storybook, or mentoring junior developers onto agentic tooling.

Two things to prepare that the resume can't carry

- **A business metric for every AI story.** Candidate 1 was rejected specifically for this. Have a number ready for each of the two tools in Question 2 — even a rough, honestly-labeled estimate beats "it felt faster."
- **A real failure you own.** Candidate 1 had no strong example of something they wished they had done better. The observability gap in Question 2 is a good one because it is true and you learned from it. Do not soften it into a humblebrag.